

Secure Coding Review Report

Application: Python Login Script

Reviewer: Nilesh Vishwakarma

Date: 10-03-26

1. Executive Summary

This review analyzed a simple Python login script for common security vulnerabilities. Several high-risk issues were identified, including SQL injection, hardcoded credentials, and poor error handling. Addressing these weaknesses will significantly improve the application’s resilience against attacks.

2. Findings

Sr.No	Vulnerability	Description	Risk Level
1	Hardcoded Credentials	Username and password stored directly in source code.	High
2	SQL Injection	User input concatenated into SQL queries without sanitization.	Critical
3	Lack of Input Validation	No checks on input format, length, or type.	Medium
4	Insecure Error Handling	Error messages expose database details.	Medium

3. Recommendations

1. Remove Hardcoded Credentials

- Store secrets in environment variables or secure vaults.
- Example: `os.getenv("DB_PASSWORD")`

2. Use Parameterized Queries

- Replace string concatenation with safe query execution.

```
cursor.execute("SELECT * FROM users WHERE username=? AND password=?", (user, password))
```

3. Implement Input Validation

- Enforce alphanumeric usernames and maximum length limits.
- Reject unexpected or malformed input.

4. Secure Error Handling

- Log detailed errors internally.
- Display generic error messages to users (e.g., "Login failed. Please try again.").

5. Adopt OWASP Secure Coding Practices

- Regularly run static analysis tools (e.g., Bandit for Python).
- Apply least privilege principles for database accounts.

4. Remediation Steps

- Refactor insecure code to remove vulnerabilities.
- Configure environment variables for sensitive data.
- Update database queries to use parameterized execution.
- Add validation functions for user input.
- Implement centralized error logging with sanitized user-facing messages.
- Conduct regular code reviews and automated scans.

5. Conclusion

- By implementing the recommended changes, the application will be protected against common attack vectors such as SQL injection and credential leaks. Continuous monitoring and adherence to secure coding standards are essential for maintaining long-term security.